removed during segment merge. Updates are supported by deletions and insertions. By default, Milvus builds indexes only for large segments (e.g., > 1GB) but users are allowed to manually build indexes for segments of any size if necessary. Both index and data are stored in the same segment. Thus, the segment is the basic unit of searching, scheduling, and buffering.

Milvus offers snapshot isolation to make sure reads and writes share a consistent view and do not interfere with each other. We present the details of snapshot isolation in Sec. 5.2.

2.4 Storage Management

As mentioned in Sec. 2.1, each entity is expressed as one or more vectors and optionally some attributes. Thus, each entity can be regarded as a row in an entity table. To facilitate query processing, Milvus physically stores the entity table in a columnar fashion.

Vector storage. For single-vector entities, Milvus stores all the vectors continuously without explicitly storing the row IDs. In this way, all the vectors are sorted by row IDs. Given a row ID, Milvus can directly access the corresponding vector since each vector is of the same length. For multi-vector entities, Milvus stores the vectors of different entities in a columnar fashion. For example, assuming that there are three entities (A, B, and C) in the database and each entity has two vectors $\mathbf{v}_1$ and $\mathbf{v}_2$, then all the $\mathbf{v}_1$ of different entities are stored together and all the $\mathbf{v}_2$ are stored together. That is, the storage format is $\{A.\mathbf{v}_1, B.\mathbf{v}_1, C.\mathbf{v}_1, A.\mathbf{v}_2, B.\mathbf{v}_2, C.\mathbf{v}_2\}$.

Attribute storage. The attributes are stored column by column. In particular, each attribute column is stored as an array of $\langle key, value \rangle$ pairs where the *key* is the attribute value and *value* is the row ID, sorted by the *key*. Besides that, we build skip pointers (i.e., min/max values) following Snowflake [16] as indexing for the data pages on disk. This allows efficient point query and range query in that column, e.g., price is less than $100.

Bufferpool. Milvus assumes that most (if not all) data and index are resident in memory for high performance. If not, it relies on an LRU-based buffer manager. In particular, the caching unit is a segment, which is the basic searching unit as explained in Sec. 2.3.

Multi-storage. For flexibility and reliability, Milvus supports multiple file systems including local file systems, Amazon S3, and HDFS for the underlying data storage. This also facilitates the deployment of Milvus in the cloud.

2.5 Heterogeneous Computing

Milvus is highly optimized for the heterogeneous computing platform that includes CPUs and GPUs. Sec. 3 presents the details.

2.6 Distributed System

Milvus can function as a distributed system deployed across multiple nodes. It adopts modern design practices in distributed systems and cloud systems such as storage/compute separation, shared storage, read/write separation, and single-writer-multi-reader. Sec. 5.3 explains more.

3 HETEROGENEOUS COMPUTING

In this section, we present the optimizations for Milvus to best leverage the heterogeneous computing platform involving both CPUs and GPUs to achieve high performance.

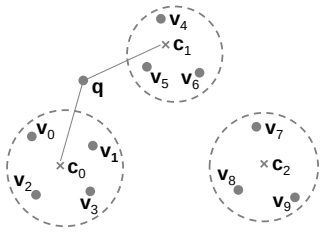

Figure 2: An example of quantization

As explained in Sec. 2.2, Milvus mainly supports quantization-based indexes (including IVF_FLAT [3, 33, 35], IVF_SQ8 [3, 35], and IVF_PQ [3, 22, 33, 35]) and graph-based indexes (including HNSW [49] and RNSG [20]). In this section, we use quantization-based indexes to illustrate our optimizations because they consume much less memory and are much faster to build index while achieving decent query performance when compared to graph-based indexes [65, 68]. Note that many optimizations (such as SIMD and GPU optimizations) can be applied to graph-based indexes.

3.1 Background

Before diving into optimizations, we explain vector quantization and quantization-based indexes. The main idea of vector quantization is to apply a quantizer z to map a vector $\mathbf{v}$ to a codeword $z(\mathbf{v})$ chosen from a codebook C [33]. The K-means clustering algorithm is commonly used to construct the codebook C where each codeword is the centroid and $z(\mathbf{v})$ is the closest centroid to $\mathbf{v}$. Figure 2 shows an example of 10 vectors ($\mathbf{v}_0$ to $\mathbf{v}_9$) of three clusters with centroids being $\mathbf{c}_0$ to $\mathbf{c}_2$, then $z(\mathbf{v}_0)$, $z(\mathbf{v}_1)$, $z(\mathbf{v}_2)$, or $z(\mathbf{v}_3)$ is $\mathbf{c}_0$.

Quantization-based indexes (such as IVF_FLAT [3, 33, 35], IVF_SQ8 [3, 35], and IVF_PQ [3, 22, 33, 35]) use two quantizers: coarse quantizer and fine quantizer. The coarse quantizer applies the K-means algorithm (e.g., K is 16384 in Milvus and Faiss [3]) to cluster vectors into K buckets. And the fine quantizer encodes the vectors within each bucket. Different indexes may use different fine quantizers. IVF_FLAT uses the original vector representation; IVF_SQ8 uses a compressed representation for the vectors by adopting one-dimensional quantizer (called "scalar quantizer") to compress a 4-byte float value to a 1-byte integer; and IVF_PQ uses product quantization that splits each vector into multiple sub-vectors and applies K-means for each sub-space.

Query processing (of a query $\mathbf{q}$) over quantization-based indexes takes two steps: (1) Find the closest n_{probe} buckets (or clusters) based on the distance between $\mathbf{q}$ and the centroid of each bucket. For example, assuming n_{probe} is 2 in Figure 2, then the closest two buckets of $\mathbf{q}$ are centered at $\mathbf{c}_0$ and $\mathbf{c}_1$. The parameter n_{probe} controls the tradeoff between accuracy and performance. Higher n_{probe} produces better accuracy but worse performance. (2) Search within each of the n_{probe} relevant buckets based on different fine quantizers. For example, if the index in Figure 2 is IVF_FLAT, then it needs to scan the vectors $\mathbf{v}_0$ to $\mathbf{v}_6$ in the two buckets.

3.2 CPU-oriented Optimizations

3.2.1 Cache-aware Optimizations in Milvus

The fundamental problem for query processing over quantization-based indexes is that, given a collection of m queries $\{\mathbf{q}_1, \mathbf{q}_2, ..., \mathbf{q}_m\}$ and a collection of n data vectors $\{\mathbf{v}_1, \mathbf{v}_2, ..., \mathbf{v}_n\}$, how to quickly

find for each query $\mathbf{q}_i$ its top-k similar vectors? In practice, users can submit batch queries so that $m \geq 1$.

This operation happens in finding the relevant buckets as well as searching within each relevant bucket. The original implementation in Facebook Faiss [3], which Milvus is built on top of, is inefficient because it incurs many CPU cache misses as explained below. Thus, Milvus develops an optimized approach to significantly reduce data movement between main memory and CPU caches.

Original implementation in Facebook Faiss [3]. Faiss uses the OpenMP multi-threading to process queries in parallel. Each thread is assigned to work on a single query at a time. The thread is released (for next query) once the current task is finished. Each task compares $\mathbf{q}_i$ with all the n data vectors and maintains a k-sized heap to store the results.

The above solution in Faiss has two performance issues: (1) It incurs many CPU cache misses, because for each query the entire data needs to be streamed through CPU caches and cannot be reused for the next query. Thus, each thread accesses m/t times of the entire data where t is the total number of threads. (2) It cannot fully leverage multi-core parallelism when the batch size m is small.

Optimizations in Milvus. Milvus develops two ideas to tackle the issues. First, it reuses the accessed data vectors as much as possible for multiple queries to minimize CPU cache misses. Specifically, it optimizes for reducing L3 cache misses because the penalty to access memory is high and also L3 cache size (typically 10s MB) is much bigger than L1/L2 cache, leaving more room for optimizations. Second, it uses fine-grained parallelism that assigns threads to data vectors instead of query vectors to best leverage multi-core parallelism, because the data size n is usually much bigger than the query size m in practice.

Figure 3 shows the overall design. Specifically, let t be the number of threads, then each thread T_i is assigned $b = n/t$ data vectors:[4] $\{\mathbf{v}_{(i-1)*b}, \mathbf{v}_{(i-1)*b+1}, ..., \mathbf{v}_{i*b-1}\}$. Milvus then partitions the m queries into query blocks of size s such that each query block (together with its associated heaps) can always fit in the L3 CPU cache. We decide s later on in Equation (1). Here we assume that m is divisible by s. Milvus computes the top-k results of each query block at a time with multiple threads. Whenever each thread loads its assigned data vectors to L3 cache, they will be compared against the entire query block (with s queries) in the cache. To minimize the synchronization overhead, Milvus assigns a heap per query per thread. In particular, assuming the i-th query block $\{\mathbf{q}_{(i-1)*s}, \mathbf{q}_{(i-1)*s+1}, ..., \mathbf{q}_{i*s-1}\}$ is in cache, Milvus dedicates the heap $H_{r-1,j-1}$ for the j-th query $\mathbf{q}_{(i-1)*s+j-1}$ on the r-th thread T_{r-1}. Thus, the results of a query $\mathbf{q}_i$ are spread over t threads of heaps. Thus, it needs to merge the heaps of each thread to obtain the final top-k results.

Next, we discuss how to determine the query block size s such that s queries and their associated heaps can always fit in L3 cache. Let d be the dimensionality, then the size of each query is $d \times$ sizeof(float). Since each heap entry contains a pair of vector ID and similarity, then the total size of the heaps (per query) is $t \times k \times$ (sizeof(int64) + sizeof(float)) where t is the number of threads. Thus, s is computed as follows:

$$s = \frac{\text{L3's cache size}}{d \times \text{sizeof(float)} + t \times k \times (\text{sizeof(int64)} + \text{sizeof(float)})}. \quad (1)$$

[4]We assume that n is divisible by t.

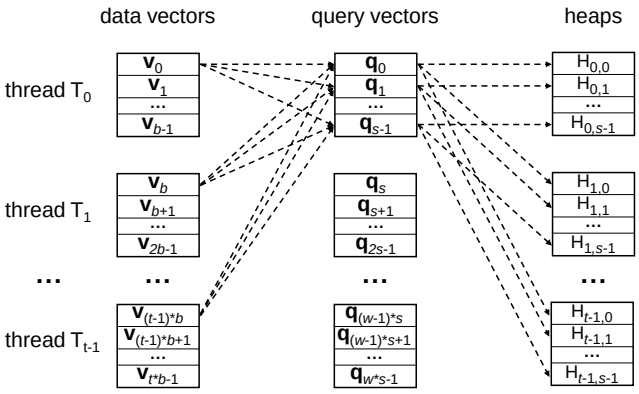

Figure 3: Cache-aware design in Milvus

In this way, each thread only accesses $m/(s*t)$ times of the entire data, which is s times smaller than the original implementation in Facebook Faiss [3]. Experiments (Sec. 7.4) show that this improves performance by a factor of 1.5× to 2.7×.

3.2.2 SIMD-aware Optimizations in Milvus

Modern CPUs support increasingly wider SIMD instructions. Thus, it is not surprising that Facebook Faiss [3] implements SIMD-aware algorithms to accelerate vector similarity search. We make two engineering optimizations in Milvus: (1) Supporting AVX512; and (2) Automatic SIMD-instruction selection.

Supporting AVX512. Faiss [3] does not support AVX512, which is now available in mainstream CPUs. Thus, we extend the similarity computing function with AVX512 instructions, such as `_mm512_add_ps`, `_mm512_mul_ps`, and `_mm512_extractf32x8_ps`. Now Milvus supports SIMD SSE, AVX, AVX2, and AVX512.

Automatic SIMD-instruction selection. Milvus is designed to work well on a wide spectrum of CPU processors (both on-premises and cloud platforms) with different SIMD instructions (e.g., SIMD SSE, AVX, AVX2, and AVX512). Thus the challenge is, given a single piece of software binary (i.e., Milvus), how to make it automatically invoke the suitable SIMD instructions on any CPU processor? Faiss [3] does not support it and users need to manually specify the SIMD flag (e.g., "-msse4") during compilation time. In Milvus, we take a considerable amount of engineering effort to refactor the codebase of Faiss. We factor out the common functions (e.g., similarity computing) that rely on SIMD accelerations. Then for each function, we implement four versions (i.e., SSE, AVX, AVX2, AVX512) and put each one into a separated source file, which is further compiled individually with the corresponding SIMD flag. During runtime, Milvus can automatically choose the suitable SIMD instructions based on the current CPU flags and then link the right function pointers using hooking.

3.3 GPU-oriented Optimizations

GPU is known for vast parallelism and Faiss [3] supports GPU for query processing over vector data. Milvus enhances Faiss in two aspects: (1) Supporting bigger k in the GPU kernel; (2) Supporting multi-GPU devices.

Supporting bigger k in GPU kernel. The original implementation in Faiss [3] does not support top-k query processing where k is greater than 1024 due to the limit of shared memory. But many

application such as video surveillance and recommender systems may need bigger k for further verification or re-ranking [69, 71].

Milvus overcomes this limitation and supports k up to 16384 although technically Milvus can support any k.[5] When k is larger than 1024, Milvus executes the query in multiple rounds to cumulatively produce the final results. In the first round, Milvus behaves the same as Faiss and gets the top 1024 results. For the second and later rounds, Milvus first checks the distance of the last result (denoted as d_l) in the previous round. Apparently, d_l is so far the largest distance in the partial results. To handle vectors with equivalent distance to the query, Milvus also records vector IDs in the result whose distances are equal to d_l. Then Milvus filters out vectors whose distances are smaller than d_l or IDs are recorded. From the remaining data, Milvus gets the next 1024 results. By doing so, Milvus ensures that results in previous rounds will not appear in the current round. After that, the new results are merged with the partial results obtained in earlier rounds. Milvus processes the query in a round-by-round fashion until a sufficient number of results are collected.

Supporting multi-GPU devices. Faiss [3] supports multiple GPU devices since they are usually found in modern servers. But Faiss needs to declare all the GPU devices in advance during compilation time. That means if the Faiss codebase is compiled using a server with c GPUs, then the software binary can only be running in a server that has at least c GPUs.

Milvus overcomes this limitation by allowing users to select any number of GPU devices during *runtime* (instead of compilation time). As a result, once the Milvus codebase is compiled into a software binary, it can run at any server. Under the hood, Milvus introduces a segment-based scheduling that assigns segment-based search tasks to the available GPU devices. Each segment can only be served by a single GPU device. This is particularly a good fit for the cloud environment with dynamic resource management where GPU devices can be elastically added or removed. For example, if there is a new GPU device installed, Milvus can immediately discover it and assign the next available search task to it.

3.4 GPU and CPU Co-design

In this mode, the GPU memory is not large enough to store the entire data. Facebook Faiss [3] alleviates the problem by using a low-footprint compressed index (called IVF_SQ8 [3])[6] and moving data from CPU memory to GPU memory (via PCIe bus) on demand. However, we find that there are two limitations: (1) The PCIe bandwidth is not fully utilized, e.g., our experiments show that the measured I/O bandwidth is only 1~2GB/s while PCIe 3.0 (16x) supports up to 15.75GB/s. (2) It is not always beneficial to execute queries on GPU (than CPU) considering the data transfer.

Milvus develops a new index called SQ8H (where 'H' stands for hybrid) to address the above limitations (Algorithm 1).

Addressing the first limitation. We investigate the codebase of Faiss and figure out that Faiss copies data (from CPU to GPU) bucket by bucket, which underutilizes the PCIe bandwidth since each bucket can be small. So the natural idea is to copy multiple buckets simultaneously. But the downside of such multi-bucket-copying is the handling of deletions where Faiss uses a simple in-place update approach because each bucket is copied (and stored) individually. Fortunately, deletions (and updates) are easily handled in Milvus since Milvus adopts an efficient LSM-based out-of-place approach (Sec. 2.3). As a result, Milvus improves the I/O utilization by copying multiple buckets if possible (line 3 of Algorithm 1).

Algorithm 1: SQ8H

```
1 let n_q be the batch size;
2 if n_q ≥ threshold then
3     run all the queries entirely in GPU (load multiple buckets
      to GPU memory on the fly);
4 else
5     execute the step 1 of SQ8 in GPU: finding n_probe buckets;
6     execute the step 2 of SQ8 in CPU: scanning every relevant
      bucket;
```

Addressing the second limitation. We observe that GPU outperforms CPU only if the query batch size is large enough considering the expensive data movement. That is because more queries make the workload more computation-intensive since they search the same data. Thus, if the batch size is bigger than a threshold (e.g., 1000), Milvus executes all the queries in GPU and loads necessary buckets if GPU memory is insufficient (line 2 of Algorithm 1). Otherwise, Milvus executes the query in a hybrid manner as follows. As mentioned in Sec. 3.1, there are two steps for searching quantization-based indexes: finding n_{probe} relevant (closest) buckets and scanning each relevant bucket. Milvus executes step 1 in GPU and step 2 in CPU because we observe that step 1 has a much higher computation-to-I/O ratio than step 2 (line 5 and 6 in Algorithm 1). That is because in step 1, all the queries compare against the *same* K centroids to find n_{probe} nearest buckets, and also the K centroids are small enough to be resident in the GPU memory. By contrast, data accesses in step 2 are more scattered since different queries do not necessarily access the same buckets.

4 ADVANCED QUERY PROCESSING

4.1 Attribute Filtering

As mentioned in Sec. 2.1, attribute filtering is a hybrid query type that involves both vector data and non-vector data [65]. It only searches vectors that satisfy the attributes constraints. It is crucial to many applications [65], e.g., finding similar houses (vector data) whose sizes are within a specific range (non-vector data). For presentation purpose, we assume that each entity is associated with a single vector and a single attribute since it is straightforward to extend the algorithms to multiple attributes. We defer multi-vector query processing to Sec. 4.2.

Formally, each such query involves two conditions C_A and C_V where C_A specifies the attribute constraint and C_V is the normal vector query constraint that returns top-k similar vectors. Without loss of generality, C_A is represented in the form of $a >= p_1$ && $a <= p_2$ where a is the attribute (e.g., size, price) and p_1 and p_2 are two boundaries of a range condition (e.g., $p_1 = 100$ and $p_2 = 500$).

There are several approaches to solve attribute filtering as recently studied in AnalyticDB-V [65]. In Milvus, we implement those

[5] In Milvus, we purposely limit k to 16384 to prevent large data movement over networks. Also, that number is sufficient for the applications we have seen so far.

[6] Note that IVF_SQ8 takes 1/4 the space of IVF_FLAT while losing only 1% recall. But the design principle and optimizations in Sec. 3.4 can be applicable to other quantization-based indexes such as IVF_FLAT and IVF_PQ.